UNIT – II: Supervised Learning – I:

Linear and Non-Linear examples – Multi–Class & Multi-Label classification – Linear Regression – Multiple Linear Regression – Naïve Bayes Classifier – Decision Trees – ID3 – CART – Error bounds.

Linear Models

- Models that assume a **linear relationship** between inputs and outputs.
- Decision boundary: Straight line (2D) or hyperplane (higher dimensions).

Examples:

1. **Linear Regression**
   - Predicting house prices based on area.
2. **Logistic Regression**
   - Email spam detection.
3. **Linear SVM**
   - Classifying two linearly separable groups.

Non-Linear Models

- Models that can capture **complex, non-linear relationships** between inputs and outputs.
- Decision boundary: Curved or more flexible shapes.

Examples:

1. **Decision Trees**
   - Predicting if a person will buy a product based on age and income.
2. **Neural Networks**
   - Image classification (e.g., cats vs. dogs).
3. **SVM with RBF kernel**
   - Handwritten digit recognition.

Multi-Class vs. Multi-Label Classification

Aspect	Multi-Class Classification	Multi-Label Classification
Definition	Each instance belongs to **one class** out of **multiple classes**.	Each instance can belong to **multiple classes** simultaneously.
Output	Single class per instance.	A set of classes per instance.
Example	Classifying animals as **cat**, **dog**, or **bird** (only one).	Predicting movie genres like **Action**, **Comedy**, and **Drama** (can be multiple).
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score.	Hamming Loss, Precision@k, Subset Accuracy, F1-Score.
Common Algorithms	Softmax Logistic Regression, SVM (one-vs-all).	Neural Networks (Sigmoid outputs), Binary Relevance.
Applications	Digit recognition (0–9), Sentiment analysis (Positive, Neutral, Negative).	Multi-topic text classification, Medical diagnosis predicting multiple conditions.

Challenges in Multi-Class Classification

1. **Class Imbalance**: Some classes may dominate, leading to poor performance on underrepresented classes.
2. **Overfitting**: The model may memorize the training data but fail to generalize.
3. **Misclassification**: Similar or overlapping classes can cause confusion.
4. **Scalability**: Increasing classes makes the model more complex.

Challenges in Multi-Label Classification

1. **Label Correlation**: Some labels may be correlated, complicating predictions.
2. **Imbalanced Labels**: Rare labels may not be predicted well.
3. **Evaluation Complexity**: Standard accuracy metrics are less useful with multiple labels.
4. **Large Label Space**: High-dimensional output space can complicate the model.

Regression

- **Definition**: A method for understanding the relationship between independent variables/features and a dependent variable/outcome.
- **Type**: Supervised machine learning technique used to predict continuous values.
- **Application**: Used in problems where the output variable is real or continuous (e.g., temperature, age, salary, price).

Why Regression Analysis?

1. Identifies relationships between variables.
2. Predicts trends and continuous outcomes.
3. Helps in forecasting and determining factor importance.

Types of Regression:

1. **Linear Regression**: Predicts outcomes using a straight-line relationship.
    - **Simple Linear Regression**: One independent variable.
    - **Multiple Linear Regression**: Multiple independent variables.
2. **Logistic Regression**: For binary outcomes.
3. **Polynomial Regression**: Fits non-linear relationships.
4. **Support Vector Regression**: For complex data structures.
5. **Decision Tree and Random Forest Regression**: For non-linear, tree-structured data.
6. **Ridge and Lasso Regression**: For regularization to handle multicollinearity.

Objective: Find the best-fit line by minimizing errors (difference between actual and predicted values).

Key Terminologies:

1. **Dependent Variable**: The variable being predicted (target).
2. **Independent Variable**: Factors influencing the dependent variable (predictors).

3. **Outliers**: Extreme values that may affect model accuracy.
4. **Multicollinearity**: High correlation between predictors, which affects model interpretation.
5. **Underfitting**: Poor model performance on both training and test data.
6. **Overfitting**: Good performance on training data but poor on test data.

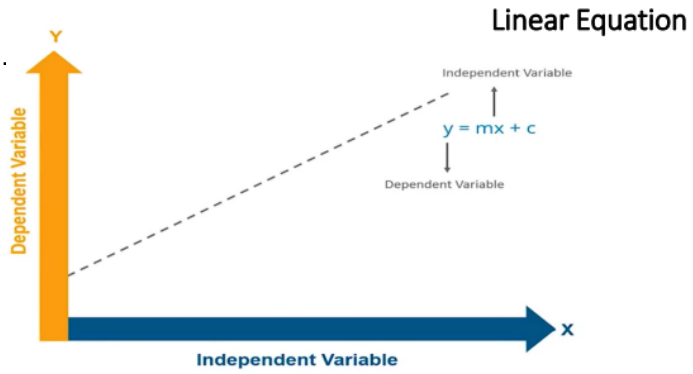
Simple Linear Regression

Purpose:

- Predict the relationship between two variables using a linear equation.
- Commonly used for predictive analysis (e.g., real estate prices, stock prices, sales revenue).

Variables:

- **Independent variable (X)**: Predictor variable.
- **Dependent variable (Y)**: Outcome variable.



Equation:

$Y=\beta0+\beta1 \cdot X+\epsilon$ or $Y=mx+c$

Where:

- **X**: Independent variable value.
- **Y**: Dependent variable value.
- **β0**: Intercept (value of Y when X=0).
- **β1**: Slope (regression coefficient, change in Y for each unit change in X).
- **ϵ**: Random error term (due to the line not perfectly fitting the data).

Relationship Measure:

- **Correlation Coefficient**: Measures the strength of the relationship between X and Y, ranging from -1 (perfect negative) to 1 (perfect positive).

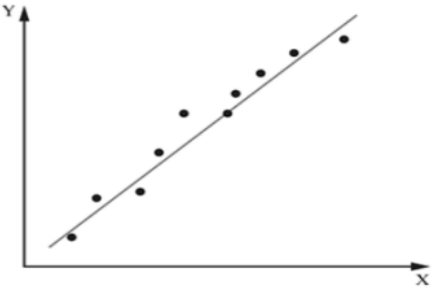
Linear Equation:

- **Equation**: Y=mx+c
  - **Slope (m)**: Change in Y / Change in X (Rise/Run)
  - **Rise**: Change in Y-axis (Y2−Y1)
  - **Run**: Change in X-axis (X2−X1)

In **linear regression**, the **slope** (m or β1) of the line describes the relationship between the independent variable (X) and the dependent variable (Y):

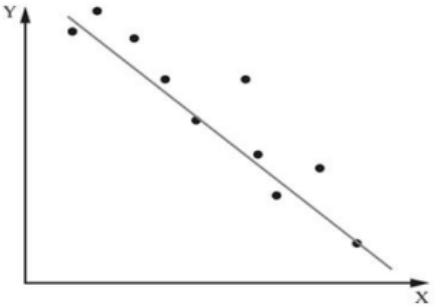
Positive Slope

- **Definition**: The slope is positive when m>0.
- **Indicates**: As X increases, Y also increases.



Negative Slope

- **Definition**: The slope is negative when m<0.
- **Indicates**: As X increases, Y decreases.



Performance Metrics for Regression

1. Mean Absolute Error (MAE)

- **Definition**: Measures the average absolute difference between actual and predicted values.
- **Formula**:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|Y_i - \hat{Y}_i|$$

- A lower MAE indicates better model performance.

2. Mean Squared Error (MSE)

- **Definition**: Measures the average of squared differences between actual and predicted values.
- **Formula**:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2$$

- A smaller MSE indicates better model performance.

3. Root Mean Squared Error (RMSE)

- **Definition**: Square root of MSE; provides error in the same unit as the dependent variable.
- **Formula**:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- Lower RMSE values indicate better model performance.

4. R^2 Score (Coefficient of Determination)

- **Definition**: Proportion of variance in the target variable explained by the model.
- **Formula**:

$$R^2 = 1 - \frac{\sum(Y_i - \hat{Y}_i)^2}{\sum(Y_i - \bar{Y})^2}$$

- **Range**: 0≤R2≤1 (or negative for poorly performing models).

Significance:

- Indicates the "goodness of fit" of a model.
- Values range between 0 and 1, where a higher value suggests better performance.

6. Adjusted R^2

- **Definition**: Adjusted version of R^2 that accounts for the number of predictors.
- **Formula**:

$$\text{Adjusted } R^2 = 1 - \left(1 - R^2\right) \frac{n-1}{n-k-1}$$

UNDERSTANDING LINEAR REGRESSION
https://www.slideshare.net/slideshow/linear-regression-algorithm-linear-regression-in-python-machine-learning-algorithm-edureka/118299696#12
PROBLEMS:
1.https://www.youtube.com/watch?v=QcPycBZomac&t=218s
2.https://www.youtube.com/watch?v=etSPUoNdIGQ
3.https://www.youtube.com/watch?v=phx4SRuQzUg&t=2s
4.https://www.youtube.com/watch?v=OXMATSKUlRY

Multiple Linear Regression

Multiple Linear Regression (MLR) is a statistical technique used to predict the value of a continuous dependent variable (Y) based on two or more independent variables (X1,X2,...,Xn). This approach helps identify the relationship between the dependent variable and predictors.

General Equation

The equation for a Multiple Linear Regression model is:

$Y=\beta0+\beta1X1+\beta2X2+\cdots+\beta nXn+\epsilon$

Where:

- Y: Dependent variable (outcome to predict).
- β0: Intercept (value of Y when all Xi=0).
- β1,β2,…,βn: Regression coefficients (measure the change in Y for a unit change in Xi, holding other variables constant).
- X1,X2,…,Xn: Independent variables (predictors).
- ϵ: Error term (difference between predicted and actual Y).

EX https://www.youtube.com/watch?v=_OyKjstWe80

Ex 2https://www.youtube.com/watch?v=7USXf1Kiq-I

Key Characteristics

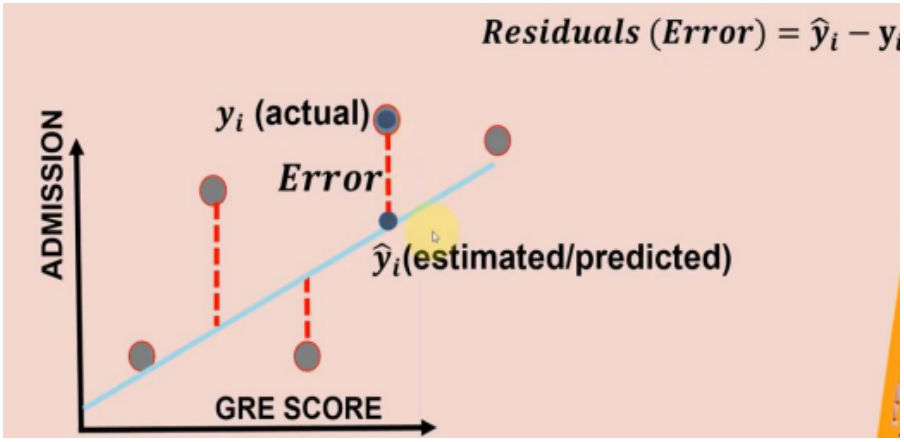
1. **Relationship**: Explains how multiple predictors influence the dependent variable.
2. **Predictive Model**: Used for prediction and hypothesis testing.

Assumptions of Linear Regression:

1. **Linearity**: The relationship between dependent and independent variables should be linear.
2. **No Multicollinearity**: Independent variables should not be highly correlated.
3. **Normality**: Residuals (errors) should be normally distributed.

Regression Line and Error:

1. **Regression Line**: A line showing the relationship between dependent and independent variables.
    - **Positive Relationship**: Both variables increase together.
    - **Negative Relationship**: One variable increases as the other decreases.
2. **Error**: The vertical distance between actual data points and the regression line.
    - **Residual/Error Formula**: Error=Actual Value−Predicted Value
    - **Goal**: Minimize the sum of squared residuals to find the best-fit line.



Applications of Multiple Linear Regression

1. **Finance**: Predicting stock prices based on market indicators.
2. **Marketing**: Determining sales based on advertising spend, product pricing, and competition.
3. **Healthcare**: Analyzing factors influencing patient outcomes.
4. **Social Sciences**: Studying relationships between socioeconomic factors and academic performance.

Key Steps in Building a Multiple Linear Regression Model

1. **Data Collection**: Gather data for dependent and independent variables.
2. **Preprocessing**: Handle missing values, remove outliers, and normalize data.
3. **Model Fitting**: Fit the regression model using techniques like Ordinary Least Squares (OLS).
4. **Evaluation**: Assess performance using metrics like R2, Adjusted R2, MAE, MSE, and RMSE.
5. **Interpretation**: Analyze regression coefficients to understand variable impacts.

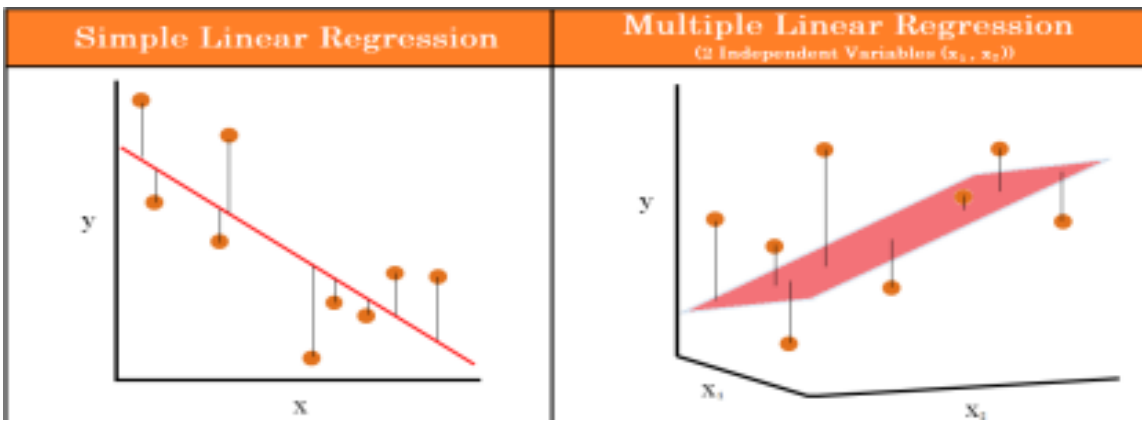
Advantages

- Allows simultaneous analysis of multiple predictors.
- Provides insight into relative importance of predictors.
- Enables testing of hypotheses about relationships.

Disadvantages

- Sensitive to multicollinearity (when predictors are highly correlated).
- Requires careful assumption checking.
- Prone to overfitting if too many predictors are included.

Simple Linear Regression	Multiple Linear Regression
One independent variable (X).	Two or more independent variables ($X_1, X_2, \ldots$).
$Y = \beta_0 + \beta_1 X + \epsilon$	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \epsilon$
Less complex and easier to interpret.	More complex due to multiple predictors.
Suitable for understanding the impact of one predictor.	Suitable for examining combined effects of multiple predictors.



Improving Accuracy of Linear Regression

1. **Shrinkage (Regularization)**:
    - **Ridge Regression**: Reduces coefficients using L2 penalty, includes all predictors.
    - **Lasso Regression**: Shrinks some coefficients to zero (L1 penalty), selects key predictors.
2. **Subset Selection**:
    - **Best Subset**: Fits all subsets of predictors (computationally expensive).
    - **Stepwise Selection**: Adds/removes predictors iteratively (forward/backward).
3. **Dimensionality Reduction**:
    - Reduces predictors by transforming them (e.g., PCA).

Aim for low **bias** (accuracy) and low **variance** (consistency) to create a robust regression model.

Classification Algorithm in Machine Learning

Classification is a fundamental **Supervised Learning technique** used to categorize data into predefined groups or labels based on training data. Unlike regression, where outputs are continuous, classification outputs are categorical, such as "Yes/No," "Spam/Not Spam," or "Cat/Dog."

What is a Classification Algorithm?

- **Definition**: A classification algorithm predicts the category or class label of new observations using labeled training data.
- **Function**: Maps an input variable (x) to a discrete output (y) where y=f(x), and y is categorical.
- **Examples**: Email spam detection, medical diagnosis, image recognition, etc.

Types of Classifications

1. **Binary Classification**:
    - Two possible outcomes.
    - Examples: "Yes/No," "Spam/Not Spam," "Male/Female."
2. **Multi-class Classification**:
    - More than two possible outcomes.
    - Examples: Types of crops, music genres, or animal species.

Learners in Classification Problems

1. **Lazy Learners**:
    - **How it works**: Stores the training dataset and performs classification only upon receiving the test dataset.
    - **Characteristics**:
        - Low training time.
        - High prediction time.
    - **Examples**:

        - **K-Nearest Neighbors (K-NN)**
        - Case-Based Reasoning.
2. **Eager Learners**:
    - **How it works**: Builds a classification model during the training phase.
    - **Characteristics**:
        - High training time.
        - Low prediction time.
    - **Examples**:
        - **Decision Trees**
        - **Naïve Bayes**
        - **Artificial Neural Networks (ANNs).**

Types of Classification Algorithms

Classification algorithms can be grouped into two categories:

1. **Linear Models**:
    - **Logistic Regression**: Predicts the probability of a data point belonging to a particular class.
    - **Support Vector Machines (SVMs)**: Finds the optimal hyperplane for class separation.
2. **Non-linear Models**:
    - **K-Nearest Neighbors (K-NN)**: Assigns class based on the majority vote of nearest neighbors.
    - **Naïve Bayes**: Based on Bayes' theorem, assumes feature independence.
    - **Decision Trees**: Splits the dataset into branches based on feature thresholds.
    - **Random Forests**: Ensemble of decision trees for improved accuracy.

Bayesian Classifiers

In many real-world applications, the relationship between the attributes (features) and the class label is non-deterministic. This means that even if the attributes of a test instance match those in the training data, the class label might still not be certain. This can be due to noisy data or unobserved factors that affect classification. For example, predicting whether a person is at risk for heart disease based on their diet and exercise habits may be uncertain due to other factors such as genetics, smoking, or alcohol consumption.

Bayes Theorem

Bayes' Theorem allows us to calculate the probability of a class given certain attributes, providing a way to reason under uncertainty. For example, in a football game scenario between two rival teams (Team 0 and Team 1), we can use Bayes' Theorem to calculate the probability of each team winning based on their past performance and whether the match is at home or away.

Naïve Bayes Classifier:

The **Naïve Bayes classifier** is a probabilistic machine learning algorithm based on **Bayes' Theorem**, with the strong (naïve) assumption that the features (attributes) are conditionally independent given the class label.

The **key idea** of Naïve Bayes is based on **Bayes' Theorem**, with a simplifying assumption that all features (attributes) are **conditionally independent** given the class label. This assumption, though often unrealistic, simplifies the computation of the probability and makes the model computationally efficient.

Working of Naïve Bayes:

1. **Conditional Independence**: Naïve Bayes assumes that each attribute Xi is independent of the others, given the class Y. This simplifies the estimation of the class-conditional probabilities P(Xi | Y).
2. **Bayes' Theorem**

   Bayes' Theorem:

   $$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

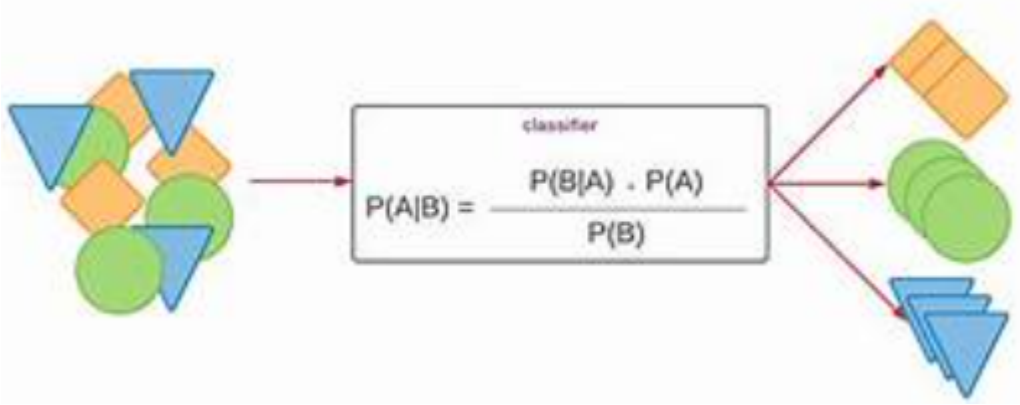
   - $P(Y|X)$: The posterior probability (probability of class Y given attributes X).
   - $P(X|Y)$: The likelihood (probability of observing attributes X given class Y).
   - $P(Y)$: The prior probability of class Y.
   - $P(X)$: The evidence (probability of the observed attributes, used as a normalizing constant).
3. **Conditional Independence Assumption**: Naïve Bayes makes the simplifying assumption that all features are independent of each other **given the class label**. This means:

   $$P(X_1, X_2, \ldots, X_n|Y) = \prod_{i=1}^{n} P(X_i|Y)$$

   This significantly simplifies the calculation of the likelihood P(X | Y), as each feature's contribution is considered independently.

4. Classification Rule: To classify a new test instance, Naïve Bayes computes the posterior probability P(Y | X) for each class Y and selects the class that maximizes this probability:

$$\hat{Y} = \arg\max_{Y} P(Y) \prod_{i=1}^{n} P(X_i|Y)$$



Types of Naïve Bayes:

1. **Multinomial Naïve Bayes**: Used for text classification problems, where the features (attributes) are discrete counts (e.g., word counts in document classification).
2. **Gaussian Naïve Bayes**: Assumes that the attributes are continuous and follow a Gaussian (normal) distribution. The likelihood $P(X_i \mid Y)$ is calculated using the Gaussian distribution.
3. **Bernoulli Naïve Bayes**: Used when features are binary (e.g., presence or absence of a feature).

Advantages:

- **Simplicity**: Easy to implement and computationally efficient.
- **Scalability**: Works well with large datasets and high-dimensional spaces (like text data).
- **Performance**: Often surprisingly effective, even with the assumption of independence between features, which doesn't hold in many real-world scenarios.

Disadvantages:

- **Independence Assumption**: The assumption that all features are independent is often unrealistic, which can lead to suboptimal performance in certain cases.
- **Zero Probability Problem**: If a feature value for a test instance is not observed in the training data, it may lead to a zero probability and thus an incorrect classification. This is often addressed using **Laplace smoothing** (adding a small constant to all probabilities).

Solved Examples

1.https://www.youtube.com/watch?v=XzSlEA4ck2I
2.https://www.youtube.com/watch?v=z8K-598fqSo
3.https://www.youtube.com/watch?v=iTqBC8GEZqs
4.https://www.youtube.com/watch?v=yRl8Yq0M3TY
5.https://www.youtube.com/watch?v=kufuBE6TJew
6.https://www.youtube.com/watch?v=Tw4U4a8VmIs

The **Gaussian Naïve Bayes** classifier assumes that the features follow a **normal (Gaussian) distribution**. This means that for a given class Y, the probability of each feature X_i is modeled using the Gaussian distribution.

Gaussian Distribution Formula:

For each feature X_i, the likelihood $P(X_i|Y)$ is given by the Gaussian (normal) distribution formula:

$$P(X_i|Y) = \frac{1}{\sqrt{2\pi\sigma_Y^2}} \exp\left(-\frac{(X_i - \mu_Y)^2}{2\sigma_Y^2}\right)$$

Where:

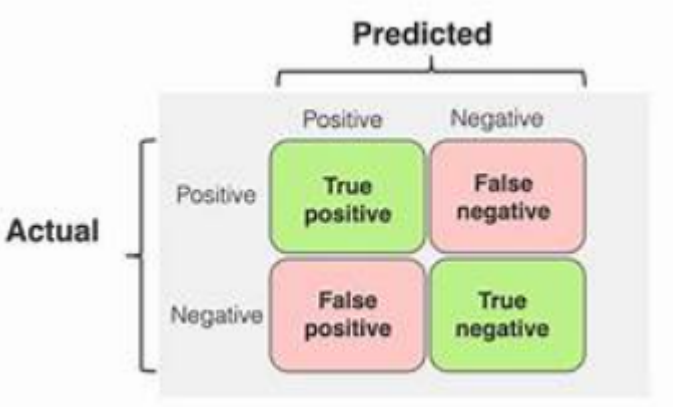
- X_i is the value of feature i.
- μ_Y is the **mean** of feature X_i for class Y.
- σ_Y^2 is the **variance** of feature X_i for class Y.

Key Evaluation Metrics for Classification Models:

In machine learning classification, model performance is typically measured using various metrics derived from a **confusion matrix**. The confusion matrix summarizes the number of correct and incorrect predictions made by a model, broken down by the actual and predicted class labels.

1. Confusion Matrix Elements:

- **True Positive (TP)**: Instances where both the predicted and actual class are positive. The model correctly identifies the positive class.
- **True Negative (TN)**: Instances where both the predicted and actual class are negative. The model correctly identifies the negative class.
- **False Positive (FP)**: Instances where the predicted class is positive, but the actual class is negative. The model incorrectly predicts a positive class (Type I error).
- **False Negative (FN)**: Instances where the predicted class is negative, but the actual class is positive. The model incorrectly predicts a negative class (Type II error).



2. Evaluation Metrics Derived from the Confusion Matrix:

- **Accuracy**: The ratio of correct predictions to the total number of predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

  This measures overall correctness but may be misleading in cases of imbalanced datasets.
- **Error Rate (ERR)**: The ratio of incorrect predictions (False Positives and False Negatives) to the total number of predictions:

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN}$$

  A lower error rate indicates a better model.
- **Precision (Positive Predictive Value - PPV)**: The ratio of correctly predicted positive instances to all predicted positive instances:

$$\text{Precision} = \frac{TP}{TP + FP}$$

  Precision is important when minimizing false positives is a priority.
- **Recall (Sensitivity or True Positive Rate - TPR)**: The ratio of correctly predicted positive instances to all actual positive instances:

$$\text{Recall} = \frac{TP}{TP + FN}$$

  Recall is crucial when minimizing false negatives is important..
- **F1 Score**: The harmonic mean of precision and recall, balancing both metrics:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

  F1 score is particularly useful when there is an imbalance between precision and recall, as it penalizes extreme values.

3. What Makes a "Good" Classifier?

A **good classification model** should maximize key metrics based on the problem's objectives:

- **Accuracy** measures overall performance but can be misleading in imbalanced datasets.
- **Precision and Recall** should be balanced depending on the use case.
    - **Precision** is important when you want to minimize false positives (e.g., spam detection).

    - **Recall** is important when you want to minimize false negatives (e.g., disease detection, fraud detection).
- **F1 Score** is a good indicator of model performance when precision and recall need to be balanced.
- **F2 Score** may be used when recall is more important than precision.

4. Example - Tumor Detection:

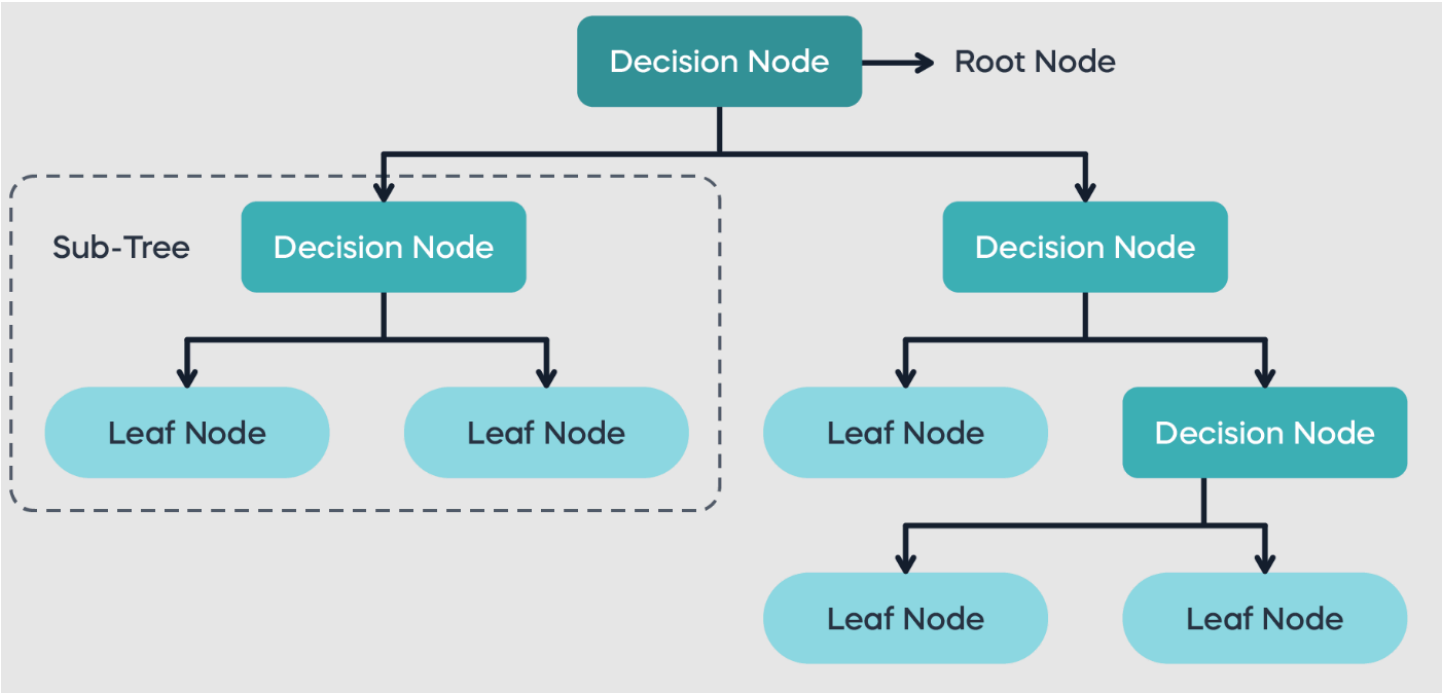
- **True Positive (TP)**: 10 patients with tumors are correctly predicted as having tumors.
- **True Negative (TN)**: 60 patients without tumors are correctly predicted as not having tumors.
- **False Positive (FP)**: 22 patients without tumors are incorrectly predicted as having tumors.
- **False Negative (FN)**: 8 patients with tumors are incorrectly predicted as not having tumors.

5. Use-Case Considerations:

- In **healthcare** (e.g., cancer detection), **recall** is typically more important than precision because false negatives (missing a diagnosis) are more harmful than false positives.
- In **e-commerce** or **fraud detection**, **precision** may be prioritized to avoid falsely identifying legitimate transactions or users as frauds.

Decision Trees:

Decision Trees (DT) are a type of supervised machine learning algorithm used for both classification and regression tasks. The goal of a decision tree is to create a model that predicts the value of a target variable based on several input features. The tree is constructed by splitting the dataset into subsets based on the most significant feature, recursively, until the subsets are pure (or nearly pure).



The two most common decision tree algorithms are **ID3 (Iterative Dichotomiser 3)** and **CART (Classification and Regression Trees)**.

1. ID3 (Iterative Dichotomiser 3)

ID3 is one of the earliest decision tree algorithms and works by constructing a decision tree top-down. The core of the ID3 algorithm involves selecting the feature that maximizes the **information gain** (IG) at each node.

Steps in ID3:

1. **Start with the entire dataset**: The root node of the tree is the entire dataset.
2. **Select the best feature**: The algorithm evaluates all features and chooses the one that best splits the data. This decision is based on **information gain**, which measures how much uncertainty is reduced by partitioning the dataset based on a feature.
   The **Information Gain (IG)** is calculated using the concept of **entropy**:

   Entropy: A measure of impurity or disorder in the data. Higher entropy means more uncertainty.

   Information Gain: The difference in entropy before and after splitting the data using a specific feature.

   - **Entropy** quantifies the uncertainty in the data:

   $$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

   - **Information Gain (IG)** for a feature A is

   $$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

   where Sv is the subset of S where feature A takes value v.
3. **Repeat recursively**: After the best feature is selected, the dataset is split, and the process is repeated on the subsets until a stopping condition is met (e.g., all instances in a subset belong to the same class, or there are no more features to split on).
4. **Assign labels to leaves**: Once the tree is built, each leaf node is assigned the majority class of instances in that leaf.

Strengths of ID3:

- Simple and easy to understand.
- Performs well on small datasets.

Weaknesses of ID3:

- **Overfitting**: The algorithm tends to overfit if the tree grows too deep.
- **Continuous Data**: ID3 doesn't handle continuous data well; it needs to be discretized.
- **Greedy**: It selects the best feature at each step without considering future consequences, which may not always lead to the best overall model.

2. CART (Classification and Regression Trees)

CART is a more generalized decision tree algorithm that works for both classification and regression problems. Unlike ID3, which uses information gain, CART uses **Gini impurity** for classification and **mean squared error (MSE)** for regression to determine the best feature to split on.

Steps in CART:

1. **Start with the entire dataset**: Similar to ID3, the process begins with the whole dataset at the root node.
2. **Split the data based on a feature**: For each feature, CART considers all possible thresholds (for continuous features) or values (for categorical features) to split the dataset. The goal is to minimize the **Gini impurity** (for classification problems) or **mean squared error (MSE)** (for regression problems) in the resulting subsets.
    - **Gini Impurity** for a dataset S is calculated as:

      $$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2$$

      where pi is the probability of class i in set S.
    - For **regression trees**, the **Mean Squared Error (MSE)** is used:

      $$\text{MSE}(S) = \frac{1}{|S|} \sum_{i=1}^{|S|} (y_i - \hat{y})^2$$

      where yi is the actual value and y^ is the predicted value for instance i.
3. **Repeat recursively**: After splitting the dataset, CART repeats the process for each subset, recursively creating new nodes until the tree reaches a stopping condition (e.g., no further improvement can be made).
4. **Prune the tree**: One key feature of CART is **pruning**, where the tree is simplified to avoid overfitting. This is done by removing branches that provide little to no predictive power.

Strengths of CART:

- Handles both classification and regression tasks.
- Can work with both continuous and categorical features.
- Includes pruning to reduce overfitting.

Weaknesses of CART:

- Pruning is computationally expensive.
- The algorithm can overfit if not properly tuned.

Solved Examples

1.https://www.youtube.com/watch?v=coOTEc-0OGw
2.https://www.youtube.com/watch?v=JO2wiZif2OM
3.https://www.youtube.com/watch?v=2A8AGfxs0D8
4.https://www.youtube.com/watch?v=zNYdkpAcP-g
5/https://www.youtube.com/watch?v=vySR1fBSpRg
6.https://www.youtube.com/watch?v=xyDv3DLYjfM

3. Error Bounds in Decision Trees

Error bounds provide an estimate of the performance of a decision tree on unseen data. They help assess how well the model will generalize.

1. Bias-Variance Tradeoff:

- **Bias** refers to the error introduced by approximating a real-world problem (which may be complex) by a simplified model. High bias typically leads to **underfitting**.
- **Variance** refers to the error introduced by the model's sensitivity to small fluctuations in the training dataset. High variance typically leads to **overfitting**.

A decision tree's complexity can affect this tradeoff:

- Shallow trees (with fewer splits) tend to have high bias and low variance (underfitting).
- Deep trees (with more splits) tend to have low bias and high variance (overfitting).

To mitigate overfitting, **pruning** is used, and various techniques like **cross-validation** are employed to estimate the model's error.

2. Error Bound Formula:

The error bound for decision trees can be expressed using the **VC-dimension (Vapnik-Chervonenkis dimension)**, a measure of the complexity of a model class:

- The error bound is given by:

  where:

  - d is the number of parameters or the depth of the tree (complexity of the model),
  - N is the number of data points (size of the training dataset).

- This suggests that as the number of data points N increases, the error decreases, but as the tree complexity d increases, the error bound becomes larger due to overfitting.

3. Generalization Error:

- **Training Error**: The error on the training data. This typically decreases as the tree becomes more complex (deeper).
- **Test Error**: The error on unseen data (test set). This error initially decreases as the tree improves but starts to increase when the tree becomes too complex and overfits the training data.

4. Cross-Validation:

Cross-validation is a powerful tool for estimating the error bound. By dividing the data into multiple folds and training/testing on each fold, the model's performance is averaged, reducing the variance in the error estimate and providing a more reliable error bound.

Entropy Calculation for Positive and Negative Proportions:

Entropy is a measure of uncertainty or impurity in a dataset. For binary classification (e.g., positive vs. negative), entropy can be calculated using the formula:

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

Where:

- p+ is the proportion of positive instances in the set S.
- p− is the proportion of negative instances in the set S.

Example:

Suppose we have a dataset with 10 instances, where 6 are positive (class "Yes") and 4 are negative (class "No").

1. **Proportions**:
   - $p_+ = \frac{6}{10} = 0.6$
   - $p_- = \frac{4}{10} = 0.4$
2. **Entropy Calculation**:

$$\text{Entropy}(S) = -0.6 \log_2(0.6) - 0.4 \log_2(0.4)$$

Using a calculator:

$$\text{Entropy}(S) = -0.6 \times (-0.736) - 0.4 \times (-1.322)$$

$$\text{Entropy}(S) = 0.4416 + 0.5288 = 0.9704$$

So, the entropy for this dataset is approximately **0.9704**.